

Agile Project Management: A Case Study of a Virtual Research Environment Development Project

Rob Procter¹, Mark Rouncefield², Meik Poschen¹, Yuwei Lin³ & Alex Vöss⁴

¹Manchester eResearch Centre, University of Manchester, Arthur Lewis Building, Manchester M13 9PL, UK (Phone: +44-161-2751381; Fax: +44-161-2751390; E-mail: rob.procter@manchester.ac.uk); ²School of Computing and Communications, Lancaster University, Lancaster, UK; ³School of Media, Music and Performance, University of Salford, Salford, UK; ⁴School of Computer Science, University of St Andrews, St Andrews, UK

Abstract. In this paper we use a case study of a project to create a Web 2.0-based, Virtual Research Environment (VRE) for researchers to share digital resources in order to reflect on the principles and practices for embedding eResearch applications within user communities. In particular, we focus on the software development methodologies and project management techniques adopted by the project team in order to ensure that the project remained responsive to changing user requirements without compromising their capacity to keep the project ‘on track’, i.e. meeting the goals declared in the project proposal within budget and on time. Drawing on ethnographic fieldwork, we describe how the project team, whose members are distributed across multiple sites (and often mobile), exploit a repertoire of coordination mechanisms, communication modes and tools, artefacts and structuring devices as they seek to establish the orderly running of the project while following an agile, user-centred development approach.

Key words: agile software project management, eResearch, Virtual Research Environment, user engagement, Web 2.0

1. Introduction

In this paper, we report findings from an in-depth study we have been conducting of a project to develop myExperiment, a Virtual Research Environment (VRE).¹ The project is funded under the second phase of the UK JISC Virtual Research Environments programme.² Its aim is to provide researchers with an integrated environment for publishing and sharing digital resources associated with their research—and, in particular, scientific workflows³—through increasingly familiar social networking and collaboration features (De Roure et al. 2007).

The challenges for embedding VREs within the everyday practices and routines of researchers are numerous and formidable (Vöss and Procter 2009). For example, given the innovative nature of such tools and their potentially intimate

relationship with researchers' working practices, their requirements are likely to be initially vague, subject to change as users begin to explore the potential uses, while needing to accommodate diversity. Further, the available technologies are evolving rapidly. In order to meet such challenges, the myExperiment project team have chosen to follow a so-called 'agile' approach to software development and project management. Their motivation for this choice has been to afford a balance between structured orderliness that ensures the project as a whole is achievable and a capacity for improvisation that enables responsiveness to new and changing user requirements, and exploitation of a rapidly expanding array of Web 2.0-based components and technologies.

The aim of our study, then, has been to explore and understand the 'lived work' of software project management in the context of an application domain in which rapid development and strong user engagement are generally acknowledged as being essential for success. We are especially interested in explicating how the project is managed so as to maintain rich user engagement in the face of uncertain and evolving requirements, to exploit the malleability of Web 2.0 technologies for rapid implementation, while keeping the project on 'track', i.e. meeting the goals declared in the project proposal, and doing so within budget and on time. Our interest then is in 'agile management'; documenting how the flexibility and responsiveness encouraged by 'agile' approaches (and facilitated by ICT tools) can be combined with management requirements. These include requirements for project coordination, such as measuring progression, identifying and meeting targets and so on, as well as the organizational rationale and consequences of 'perpetual beta'; i.e. where new features are regularly released (Morris 2006).

For the myExperiment project team, whose members are distributed across multiple sites (and often mobile), making an agile approach work in practice means: relying on a repertoire of mechanisms, deployed at different frequencies (daily, weekly, monthly), involving different combinations of team members (developers, managers, users), focusing on different project aspects and time-scales. It also involves using a variety of communication modes and tools, artefacts and structuring devices as the project partners seek to establish and maintain the orderly running of the project.

In this paper, we will examine how the myExperiment project team's pursuit of a 'perpetual beta' is managed day-to-day and unravel the agile management contextualised in myExperiment. We focus on how the team actually 'does agility' when dealing with the everyday practicalities and contingencies of distributed working and users' evolving requirements. We use data gathered from logs of daily project team Skype instant messaging (text) 'chats' and email postings to examine how, day-to-day, the team accomplishes the orderly character of project work. Here, we make use of the analytic framework of Button and Sharrock (1996), namely: phasing, orienting to the project as a totality, the methodic handling of tasks, making sure the

documentation gets done, and measured progression; in order to examine how various activities are deployed as a means of accomplishing the orderly character of project work.

One of the driving forces behind the adoption of methodologies for software project management has been to provide a template for practice that can be readily adopted across the software engineering community. This is a highly questionable goal, however, as it ignores the work that goes into appropriating methods and tools and using them practically, according to the purpose at hand (Russo and Stolterman 2000). Throughout the history of software, there has been much discussion in the literature as to whether methodologies for software project management represent the sequence in which software engineers actually work, and whether, if prescribed for projects, they actually benefit, rather than inhibit the work (Wastell 1996).

What is distinctive about agile methods is that they seek to establish principles to which software project management should aspire if it is to take user engagement seriously and deal with volatile requirements, but leave their accomplishment to the situated judgment of those involved. Our interest in this paper is to explore how this is accountably achieved in order to develop a better understanding of exactly what it means to apply such principles in everyday practice.

We begin by briefly reviewing the emergence of agile methods, setting out their key concepts and findings from various studies of their use in software projects, and outlining the framework for the analysis of our own case study. Having set out the background to our case study, we draw on accounts of daily project meetings and email postings to discuss and explicate the significance of a range of relevant project activities. The examples we use include instances of reporting, coordinating, scheduling, selecting, prioritising, reviewing, problem-solving, revising, refining, and testing. We document how these appear essential for the project team's ability to stay on 'track' in the face of almost constant demands for changes and new features. Finally, we consider lessons learned and conclude with some general recommendations software development practices which may further the embedding of an eResearch agenda in this new and important application field.

2. Software project management

Initial solutions to the problem of how to manage software projects emerged as a means to bring order to what was seen as a chaotic process (Royce 1987). These early approaches broke software work into phases such as 'analysis', 'design', 'implementation', 'testing', implementation and so forth, and imposed a strict sequence on the order in which these activities were to be performed (Sommerville 2001). Subsequently, it became increasingly clear that such 'linear' methods did not work well in many circumstances. Consequently, while phasing was retained as a software project management device, linear sequencing was

increasingly abandoned in favour of iterative, evolutionary methods (Boehm 1988; Sommerville 2001). The introduction of iterative methods was an acknowledgement that initial specifications were often imperfect, that there would inevitably be drift between a static requirements specification and a changing world, and that repairing these problems early saved effort and money.

More recently, a profusion of approaches have emerged under the rubric of ‘agile’ methods that enshrine iteration (rather than merely accommodating it) as the key to successful software projects. Agile methods tackle the problem of uncertain requirements by making use of a short development lifecycle, based on continuous ‘loops’ through design, implementation and maintenance. There is no universal definition of agile methods, but one of the most notable examples is extreme programming (XP) (Beck 2000). Its key elements include a focus on working code and short release cycles, an incremental planning approach that allows changes to be made according to evolving circumstances and user requirements. In addition, it involves tests, code and communication to represent system structure and intent, and a commitment to ongoing design throughout the software lifecycle (Beck 2000). By delivering value and spotting problems early, and by emphasising simplicity, XP promises to deliver high quality systems and the flexibility essential to deal with changing needs. Unlike traditional, phased design methodologies, agile approaches value interactions over processes and tools, and collaboration with users over production of comprehensive documentation. Other variants of agile methods that are of particular relevance for our study include Scrum, which is described as an adaptive and flexible approach to the management of software projects (Schwaber and Beedle 2002; Abrahamsson et al. 2003).

Such agile approaches are particularly useful for developing what is now referred to as a ‘perpetual beta’, a term for software which is undergoing continuous refinement and where new features that may not be fully tested (Morris 2006). The concept of the perpetual beta is perhaps most strongly associated with Web 2.0 applications. Thus, O’Reilly Media, a publishing company whose name has become synonymous with advocacy of technical and methodological innovation, offers the following advice on implementing a perpetual beta approach:

“When devices and programs are connected to the internet, applications are no longer software artifacts, they are ongoing services. Therefore: Don’t package up new features into monolithic releases, but instead add them on a regular basis as part of the normal user experience. Engage your users as real-time testers, and instrument the service so that you know how people use the new features.”⁴

While there is plentiful evidence of agile methods being taken up within a wide variety of software development contexts, there are relatively few detailed, empirical case studies of how (or if) such methods are actually being applied in practice (Dyba and Dingsoyr 2008). This is particularly true of agile methods in the context of

distributed software projects (Hole and Moe 2008). An issue that inevitably arises for distributed projects is how project managers and their team members coordinate their activities and it is one that takes on special significance for agile methods, given their emphasis on communication and prioritisation of ‘individuals and interactions over processes and tools’ (Fowler and Highsmith 2001). Indeed, Sharp et al. (2009) claim that “*Agile teams tend to be co-located [...] and rely heavily on face-to-face communication and seemingly simple physical artefacts to support interaction*”. Investigating whether and how agile methods can be made to work in a distributed team environment is one of the aims of this present study.

In consequence, one touchstone for this study is Ramesh et al.’s (2006) ‘Can distributed software development be agile?’ which enquires into how organizations can successfully blend agile methods with distributed software development. In particular, they identify challenges relating to communication, control and trust that need to be addressed; and outline a range of practices to meet these challenges. Our interest lies in the extent to which some of the practices they identify by interview in their empirical work can also be seen in the everyday work of the myExperiment project team. These practices include: continuously adjusting the process; facilitating knowledge sharing; improving communication; building trust and trusting but verifying—practices that generally involve changes at the managerial or organizational level and thus engender potential tension between agility and plan.

It is this emphasis on the everyday, mundane work of the project that requires us to turn the work of Button and Sharrock (1994; 1996) to provide a framework for our analysis. In their studies of the organisation of collaborative design and development in software engineering, Button and Sharrock (1994; 1996) point to ‘the project’ as a visible and discoverable set of formatted organisational arrangements within and through which software engineers coordinate their work, and make it both personally and organisationally accountable. Their findings emphasise how the orderly character of software engineering work, in the face of its contingent and situated circumstances, is the achievement of participants’ situated practices of work. In other words, even in what we will refer to as ‘conventional’ software development practice, coherence is achieved not by blindly following methodologies but by adapting them to the circumstances of the project. It is important, therefore, to understand the kinds of ordering devices which participants deploy to do this:

“...much effort is expended on contriving devices which will provide ‘orderliness’ in the conduct of work and in ensuring that such devices can be implemented and enforced. These devices are meant to enable the achievement of orderly work where it requires the collaborative participation of many individuals, and may, crudely, be characterised as devices which are designed to create and support teamwork.” (Button and Sharrock 1996: 373).

Button and Sharrock (1996) document a number of devices for ensuring the orderly character of work. ‘Phasing’, for example, ensures that necessary tasks

are adequately completed, and provides for the various roles and actions associated with the interdependence of activities, as well as providing a means for the recognition of incomplete stages. The ‘methodic handling of tasks’ provides for some kind of system, some kind of order, in the confrontation and elimination of problems as they arise in the project. ‘Measured progression’ refers to procedures and devices—organisational metrics—for documenting how much of the project has been done and what remains; checking work against schedules, ‘what have we done, what do we need to do next, how far have we got’ and so on. Finally, ‘orienting to the project as a totality’ provides a method for project team members to keep each other’s progress in view and make it visible, accountable to others, obviously vital in. According to Button and Sharrock (1996),

“the perception of interrelationship between the activities of project participants and the objectives of the project are a problematic feature of project work and that a critical component of the achievement of an orderly relationship between the two is the provision of opportunities for the participants to orient to ‘the project as a whole’” (*ibid*: 378).

They rightly point out that the key to a successful project is “*not merely a matter of disposing of the individual’s work in hand, but of carrying it out as work which is done in orientation to the project’s needs, problems and objectives.*” (*ibid*: 378; emphasis added). Below, we describe how the myExperiment team works to meet this challenge.

3. Methodology

We conducted ethnomethodologically informed ethnographic observations, both in-person (Hughes et al. 1994; Randall et al. 2007) and virtual (Hine 2000), of the work of the myExperiment project team over a period of 18 months, starting in mid 2007. Ethnomethodologically-informed ethnographic investigations endeavour to make visible the real-world sociality of particular settings in order to develop an understanding of the situatedness of individual activities and of the wider work setting; highlighting interdependencies between activities, and stressing the ‘practical participation’ of individuals in the collaborative achievement of work (Suchman 1995). This choice of method is motivated by our aim is to move away from idealizations of how projects are accomplished and, instead, to promote a ‘sensitivity’ to the real-world, real time character of project activities in context. This requires attention to the specifics of the setting, showing exactly how team members engaged in highly distributed project work simultaneously engage in a range of everyday practical actions to both manage the project and move the project forward.

Our study then orient to the ‘real world’ of how distributed project work is actually done, emphasizing the meaningful and practical human activity involved in the orientation of participants to colleagues, to users, to work artefacts, and to

the technology. Ethnomethodologically-informed ethnography observes in detail everyday working practices and seeks to explicate, to describe in rich detail, the numerous, situated ways in which those practices are actually achieved, and the things that such an achievement turns upon (Crabtree et al. 2000).

In this paper, we focus on a 9 month period between October 2007 and June 2008, during which we observed (virtually) approximately 100 daily project meetings conducted using Skype chats; four monthly meetings (also conducted using Skype chats); observed (in-person) two monthly meetings conducted face-to-face; and subscribed to project mailing lists (both an internal project list for team members only and an external one for the inner circle of myExperiment users). Records of face-to-face meetings, logs of the Skype chats and mailing lists postings were collected to provide a rich project 'biography', documenting day-by-day the origins, evolution and fate of different requirements, interactions between team members and with users, the various contingencies that arose and how these were managed by the team as the project unfolded.

The data was analysed using a broadly 'grounded-approach' (Glaser and Strauss 1967), involving iteratively examining records of face-to-face meetings, Skype chat logs and mailing list postings, looking for patterns, themes or analytic tropes as they appear in the data. Based on this, we then identified a core set of issues typifying what the project team had to deal with during the period of our study. These issues form the basis of both our overall analysis of the project and our comparison with other studies.

4. The myExperiment project observed

myExperiment is a collaborative environment where scientists can safely publish their workflows and experiment plans, share them with groups and find those of others. Workflows, other digital objects and bundles (called Packs) can now be swapped, sorted and searched like photos and videos on the Web. [...] myExperiment [...] makes it really easy for the next generation of scientists to contribute to a pool of scientific methods, build communities and form relationships—reducing time-to-experiment, sharing expertise and avoiding reinvention. (myExperiment website)⁵

From the very beginning of the project, myExperiment has sought to establish itself at the forefront of efforts to realise the e-Science or eResearch vision of more collaborative and more open research practices (Hey and Trefethen 2003). myExperiment's focus has not just been on making it easier for scientists to discover and re-use information, but to help drive forward innovations in scholarly communications by providing new ways to publish and share research resources (Procter et al. 2010). To achieve this, myExperiment has borrowed heavily from social media and networking

concepts made possible by Web 2.0, and popularised by sites such as Facebook and MySpace, to create a ‘social networking site for scientists’⁶ which facilitates informal interactions and controlled publishing of research resources, such as workflows (see Figure 1). myExperiment provides facilities for anyone to browse and re-use resources (such as workflows) uploaded by other users. myExperiment members are able, in addition, to add comments about other members’ contributions, to upload their own resources and share them with others, join and form groups, and interact with other members. As of March 2011, myExperiment has over 3,000 members, most of whom are from Europe and the U.S., and more than 1,000 workflows.

Despite their evident popularity, translating social media and networking concepts that work well in a leisure context into the world of research was recognised as being a challenge by the myExperiment project management. To tackle it, it was decided to follow a user-driven, agile approach to software development from the outset. Their approach consists of six key principles which are summarised in De Roure and Goble (2008): *fit in, don’t force change; jam today and more jam tomorrow; just in time and just enough; act local, think global; enable users to add value; and design for network effects*. Derived over several years experience of developing e-Research applications and further evolved during the myExperiment project, De Roure and Goble further argue that

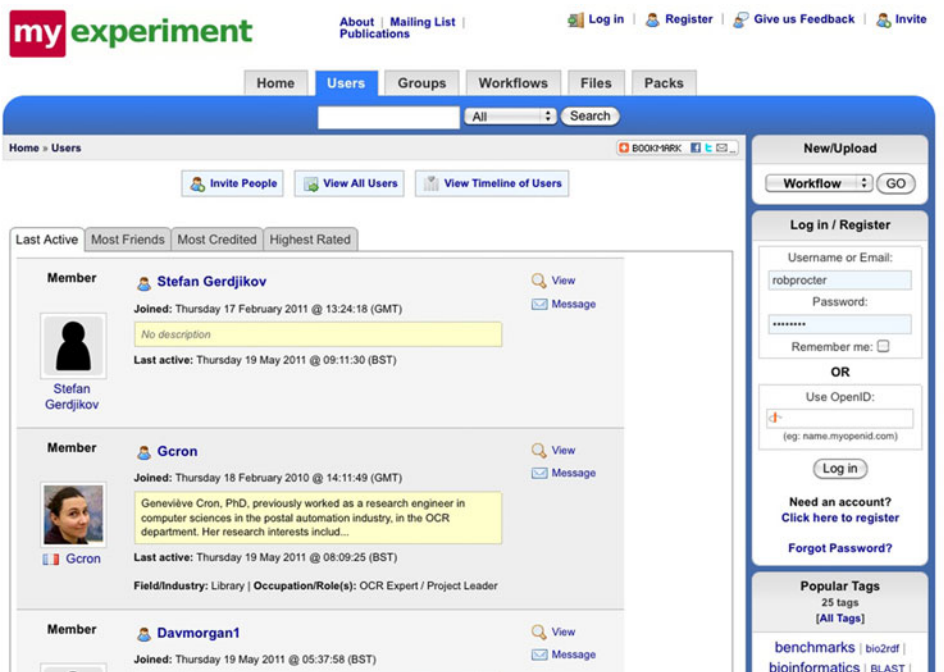


Figure 1. A section of the myExperiment VRE showing user profiles.

embedding users and developers in a project, and having them work closely together is the best way of achieving a system that will be useful for research communities.

The myExperiment project team is distributed over two sites, at Manchester and Southampton. Organisationally, the team is embedded within the myGrid project and a larger software development group responsible for delivering workflow tools, specifically, the Taverna workbench (Gil et al. 2007) whose users the project managers refer to as the 'myGrid family'.⁷ Within this 'family', there exists a kind of 'inner circle' of users on whom the myExperiment project team rely on to help drive the requirements, provide feedback and test the software implementation.

The myExperiment project relies heavily on meetings to help team members coordinate their activities and make decisions. These occur at different frequencies (daily, weekly, monthly) and involved different combinations of team members (developers, managers, users). Weekly meetings are for project management to track development and re-evaluate priorities. Monthly meetings allow a wider workflow community (e.g. developers from the myGrid project) to take part. Some meetings are conducted face-to-face, but the vast majority are computer-mediated in order to cope with the demands of distributed teamwork. Of the latter, the most important for our aim of understanding agile management and the day-to-day coordination of the project are the one-hour Skype chat team members hold daily at 5 pm to (*inter alia*): review progress; discuss and decide how to respond to user requests; review technical strategies; prioritise tasks; and schedule changes.

The project also runs two mailing lists (one open for general discussion, one limited to the core team) and a wiki (with open and secure area). The open mailing list has a dual role, serving as a mechanism for users to comment on existing features and raise new requirements, and for developers to announce progress and discuss in an open way with users how new requests will be dealt with. Those members of the inner circle of users who (like the project team) are located in Manchester or Southampton, sometimes provide feedback on myExperiment face-to-face, but all also interact frequently with the project team through the mailing list.

As we illustrate below, it is in these ways and through these means for managing the project, while involving users actively in the process, that the team succeeds in keeping the development of myExperiment moving constantly and rapidly forward as a 'perpetual beta'.

From the Skype chat logs, we can easily identify the core participants (usually 3–5 team members, including the project manager). The topics reflect the project team's daily activities, and involve informing each other of progress, making sense of problems together and solving them. The daily Skype chat usually begins with greetings and social chat, and then goes into the main topic. So, for example (Skype chat, February 15, 2008):

(Abbreviated Skype chat transcription: PM = project manager; Dev = developer)

[17:11:47] PM: hello everyone
 [17:11:56] ... i think DevB is doing his thesis
 [17:12:06] ... (becasue he has to hand it in on 18th, which is Monday!)
 [17:12:10] ... and DevC is at google
 [17:12:15] DevE: and DevB has left the building
 [17:12:32] DevA: ok
 [17:12:37] PM: shall we do updates? devA ->devE
 [17:12:43] DevA: sure
 [17:12:51] ... ok, so enactment¹ is going very well...
 [17:12:59] ... the runner/runnable model is looking very clean and generic
 [17:13:19] ... and i have connected up most of the “backend” process of enactment, with the client API², which is proving to be very useful
 [17:13:36] ... i’m now working on UI for creating runners/jobs/input forms
 [17:14:07] ... Don has given me the beginnings of his javascript³ used to generate a “list of lists” style input form... will use this as a basis for the dynamic input forms generation
 [17:14:13] PM: gr8
 [17:14:23] ... thank for progressing this!
 [17:14:27] ... it will be v important :)
 [17:14:42] DevA: i’m convinced (fool’s words maybe) that we will have enactment done before end of next week :)

Notes

¹ In the context of workflows, enactment refers to the capability to execute a workflow.

² API: an Application Program Interface defines how a piece of software interacts with software and how other programmers can use it. It is the key to building distributed software applications.

³ Javascript is a programming language used primarily for creating user interfaces to web-based applications.

⁴ Typographic and spelling errors in this and following Skype chat excerpts as in the original logs.

So, within a minute and a half people have been invited to join the chat and various greetings and social chit-chat exchanged. The project manager then gets down to the business of the meeting, which is an update on progress—“shall we do updates?” The conversation flow is fast-paced with participants very used to typing and instant-messaging. It is interesting to note that the project team prefers typing over talking, including the use of ‘textspeak’ (gr8) and emoticons, (and uncorrected miss-spellings) when using Skype. Skype chat, in this case, functions like a shared wall/whiteboard, allowing participants to draw on information from

as many sources as possible (URLs, multi-media objects, copy and paste email texts).

5. Findings

The approach to project management adopted by the myExperiment team attempts to be both planned and flexible. Planned in order to structure the project and enable tracking of its progress, and flexible in order to deal with issues as and when they materialise. The planning can be seen in two ways. First, in the project plan, consisting of: what the team are accountable for to the funder (developing a website and associated tools); a schedule of workpackages (elaborated as sets of tasks, milestones, deliverables and responsible team members); a high-level technical architecture (identifying a set of technologies, software modules and their inter-connections); and an overview of the methodology (involving iterations and consultations with users). Second, in the various mechanisms (regular meeting schedules, use of specific forms of communication which members are required to use, etc.) by which the project is routinely coordinated, moved forward and progress assessed. The flexibility is embodied in negotiations over problem-solving (e.g. the definition of the user community, or selection of reusable code). That is, as Suchman (1994) so cogently notes, the working out of a plan is a ‘situated action’ that needs to adjust to circumstances as they arise.

In the following subsections, we examine through the use of fieldwork extracts from Skype chats, how the project team dealt with a range of issues and problems that exemplify the key challenges faced during the period of our investigation: prioritising work; deciding on technical strategy; managing user engagement; diagnosing and fixing technical problems; managing user expectations; and reviewing the project’s scope. The fact that our analysis has thrown up these kinds of issues is not, in itself, surprising, as anyone familiar with software project work can probably attest. Nor are these issues the main focus of this paper; rather, the focus is to examine in some detail how the myExperiment team attempts to deal with these issues and to negotiate various tensions and challenges as part of the everyday, mundane work of doing agile, distributed software project management.

5.1. Ordering the project

In viewing ‘myExperiment’ as a ‘project’ we also wish to highlight its ordering features, documenting how the team achieve the formatted arrangements of the project and how they routinely and regularly, as an accountably mundane feature of their project meetings, display an orientation to these arrangements in the practical way they order and accomplish their work. Of course, such ordering work does not in itself automatically ensure the orderliness of work or provide

remedies for all conceivable contingencies, instead the project structure and plan is an achievement, constantly worked at, referred to and amended as the team respond to the various contingencies of their work.

All the various devices for ensuring the orderly character of work noted by Button and Sharrock (1996) are clearly noticeable in the records of myExperiment team discussions. Within myExperiment, a repertoire of coordination mechanisms and devices have been deployed to routinely and frequently cross-check the progress of the work. These include regular meetings scheduled daily (conducted by Skype chat) and monthly (conducted by Skype chat or face-face), CVS⁸ and mailing lists. And since the project team members are distributed across multiple sites (and some may also be travelling), most of these activities are made possible by the adoption of a range of communications channels and the participants' competence in moving fluidly between them.

Here is an example of a user suggestion originating on the external mailing list. On November 13, 2007, the project manager announced on myExperiment-discuss that "myExperiment has officially moved to open beta. [The developers] did the DNS update yesterday afternoon so www.myexperiment.org is now live." Users responded with feedback an hour later:

Looks good, some early feedback from people here at MIB.

- [1] Can/should myExperiment also build a Facebook app* (not a group)? This wouldn't duplicate myExperiment functionality but would provide some subset of it.
- [2] Is there a myExperiment API (I remember this came up before with the Google widget stuff)
- [3] Is there API documentation anywhere?
I think these questions might have been asked before, but I wasn't around to hear the answers...

Tracking the chronology of this suggestion shows it featuring in face-to-face management meetings, meetings between users and project team members, the project team's daily Skype chat, the developers' 'hackfest',⁹ and other brainstorming and information sharing on mailing lists, and illustrates how the project team make use of these different channels to communicate, update and prioritise their work. User requests, in the form of feedback on the current implementation and suggestions for new functionality, need to be attended to expeditiously if users' motivation to engage is to be maintained. This does not mean, however, that all such requests will be acted upon, as that might throw the project 'off track'.

Managing the tension between keeping the project 'on track' and attending to users' requests becomes visible in the practices surrounding prioritisation of tasks. The next example shows how the team attends to prioritising updating the front page (also called 'homepage'). A new version (with a screenshot) has been

distributed on the internal mailing list and this is picked up in the daily developers chat on Skype (March 11, 2008):

(Abbreviated Skype chat transcription: PM = project manager; Dev = developer)

[17:19:54] PM: let's return to that when we've discussed some other things

[17:20:05] DevA: ok

[17:20:05] PM: because i think we need to do so prioritisation

[17:20:08] DevA: yeap

[17:20:33] PM: ok, home page now, then performance

[17:20:44] PM: you'll have seen my mail to top secret [the internal mailing list]

[17:21:08] PM: saying these two things matter esp for the people who visit us after seeing nature, iece, thes, or when we do our press release

[17:21:25] PM: I'm worried we tend to focus more on existing users than new sometimes

[17:21:36] DevA: well, we did do that big users/emails work

[17:21:36] PM: so this is a bit of focus on the new!

[17:21:40] DevA: which applies to new users

[17:21:45] PM: yup

[17:21:49] DevA: but yes, front page need redoing

[17:21:59] PM: so i think the front page is a big improvement

[17:22:02] PM: and good to have it up

[17:22:12] DevA: I've spent about 90 min with users just before this

[17:22:18] DevA: and based on their feedback i have a new version

[17:22:21] PM: I'd really like to know what users would want to see there to make it inviting

[17:22:23] DevA: i've emailed to topsecret [the internal mailing list]

[17:22:28] PM: excellent

Here the project manager is orienting the developers to the need to prioritise their tasks—'ok, home page now, then performance'. The interest in the user experience, especially new user experience—I'm worried we tend to focus more on existing users than new sometimes—surfaces in their concern over the home page. Here, then the focus of the discussion is typical of agile methods' attention to delivering value to the user early in the process, and getting rapid feedback—I've spent about 90 min with users just before this and based on their feedback i have a new version'.

Aspects of the work of team management, of building a team ethos, are also clearly present. First, while the project manager visibly remains in charge of running the meeting by initiating and terminating the discussion, and stating the prioritisation principles, developers also emerge as taking on some responsibility for the direction of the project—I've emailed to topsecret.' Second, as Hochschild (1983) might suggest there is evidence of some 'emotional labour' going on here, some building of team morale by the

project manager in his praise for the work—‘the front page is a big improvement and good to have it up’, ‘excellent’, etc. This affective layer of the dialogue reminds us that project management is also a lived experience.

5.2. Deciding on technical strategy

The development of a workflow ‘enactment’ capability for the myExperiment VRE was ongoing when we started observing the daily Skype chats. Thus, we were able to have a diachronic view of the issue and the entangled socio-technical issues involved. The development of an enactment capability highlights some fundamental questions about the project, including what technologies might be used and the nature of the user community being targeted (Is there only one universal myExperiment user community or many fragmented ones?). Such questions were open issues at the start of the project and would only be answered as the project moved forward. As these questions are raised and resolved so project members are also required to orient to the project as a totality.

The issue of enactment came on to the team agenda following a feature request raised by some Taverna users to be able to execute workflows published via myExperiment without leaving the myExperiment VRE. This was accepted as being a desirable enhancement to the myExperiment VRE in that it would facilitate a greater degree of integration between the discovery of workflows and their use: the researcher would be able execute a workflow without having to leave the VRE. To provide an enactment capability, several questions need to be addressed that would have a bearing on the wider technical strategy the team would chose to follow.

Below is an excerpt from the Skype chat log on February 12, 2008 in which some of these questions are discussed and resolved. The chat, taking place when the project manager was travelling, was brief but significant:

(Abbreviated Skype chat transcription: PM = project manager; Dev = developer; XX = senior project team member)

[17:34:50] PM: i have 15mins.

[17:35:03] PM: is there anything in particular you need me for in this window ?:).

[17:35:14] PM: (I mean 15 min window !).

[17:35:35] DevA: not really ... quick update from me is that i've played with the prototype enactment code .

[17:35:44] PM: well done !.

[17:35:46] PM: saw the emails.

[17:35:50] DevA: and now going to start work on the experiment/enactor stuff.

[17:35:55] PM: you're doing the right thing.

[17:36:01] PM: gr8.

[17:36:02] DevA: ok great.

[17:36:08] DevA: i was a bit worried by XX's email:).

[17:36:21] PM: I'll shout when i think there is something useful in the grid job world that we should use rather than reinvent.
[17:36:25] PM: we're ok at the mo.
[17:36:32] DevA: ok that would be great.
[17:36:32] DevA: thanks.
[17:36:47] PM: XX is closer to grid workflows.
[17:36:54] PM: but i know all the job stuff pretty well.
[17:36:57] DevA: i purposely tried to make the jobs design simple enough to build in the timeframe now.
[17:37:01] DevA: ok.
[17:37:02] PM: exactly right.

The extract shows the participants (the project manager PM and developer DevA) discussing options for implementing the enactment capability. It highlights again the mobile way of participating, the flow of information between different communication channels (emails and Skype chat) and how these are brought to bear on issues, such as what technologies to use and whether to reinvent or re-use. The particular issue raised and quickly resolved is whether enactment implementation should adhere to so-called 'grid job' technical standards. They decide on this occasion to ignore the maxim that 'it's better to re-use than re-invent' and, instead, to opt for creating a bespoke solution, despite evident time constraints—'i purposely tried to make the jobs design simple enough to build in the timeframe now'. This decision is clearly influenced by the PM's apparently low opinion of the alternative—'I'll shout when i think there is something useful in the grid job world that we should use rather than reinvent'.¹⁰

Finally, we see in the comments of the project manager—'exactly right', 'you're doing the right thing', 'gr8', etc.—further evidence of his ongoing efforts at teambuilding and maintaining morale.

5.3. User requirements and shifting community

Throughout the period in which we observed the myExperiment project team at work, we see a constant negotiation, re-defining and re-discovering of user requirements. As noted earlier, the project team relies heavily on myGrid family users as a source of myExperiment requirements and to act as pilot users. However, the myGrid family does not necessarily represent the whole future (emerging) user community. The project team has to continue to 'imagine' their potential users (including people using different workflow editors, and from different disciplines and backgrounds), re-define the boundaries of their user community (although the local pilot users are good reference points) and recruit new users (e.g. at conferences). To do so, they (loosely) categorise potential users and build different scenarios around them. However, the boundary of the myExperiment user community would be redrawn only if other requirements can

be successfully negotiated, and subject to the constraint that existing Taverna users' needs are not compromised too much.

In the next Skype extract (February 15, 2008), two project developers address the question of whether the enactment capabilities referred to in the previous extract are being developed solely for Taverna workflow editor users (i.e. as typified by myGrid family members) or for a wider workflow community. It demonstrates project team members' use of the Skype chats to maintain awareness of the project's status, to work up a common understanding of an issue, the fluidity of—and the readiness to engage users in—decision-making processes:

(Abbreviated Skype chat transcription: Dev = Developer; YY = user)

[17:15:16] DevA: the last i did on the taverna input editor was show it to YY and the scuf¹ extension i did.

[17:15:25] DevB: i think with the runner/runnable model we might even be able to support another workflow engine soon.

[17:15:28] DevA: he wasn't happy with the modifications to scuf, so the editor ended up being generic still.

[17:15:55] DevA: i suspect that as we get the input definition into wherever it fits into, we can lose most of those "-> list" buttons etc.

[17:16:01] DevB: yeah, i think the scuf extensions instead will become annotations on input ports using the new annotation model.

[17:16:08] DevA: which would make it much easier on users using the input editor.

[17:16:11] DevA: yeah.

[17:16:19] DevA: that's the problem YY had with it.

[17:16:24] DevA: but now is the right time to do that part.

[17:16:38] DevB: I'm not sure ... because a basic principle is that there is no limit on the amount of lists of lists yuo can have for one input port.

[17:16:48] DevA: i know.

[17:17:11] DevA: but being too generic leads us into the problem where casual users have to create the structure, not just the actual inputs.

[17:17:28] DevB: to do which part ? the annotation model stuff ?.

[17:17:29] DevA: i was under the impression that we were going to give people canned example input sets.

[17:17:50] DevA: being able to say "input a" is just a string and "input b" is a list of strings.

[17:18:00] DevB: we could do ... a discussion arose today about storing example input data sets.

[17:18:13] DevA: that discussion has been had a few times over the project.

[17:18:18] DevB: yeap.

[17:18:21] DevB: i expect so.

[17:18:33] DevB: what we'll do is have the users play with what we have and then see what they ask for.

[17:18:35] DevA: anyway, i think we most of the pieces to fit together.

[17:18:43] DevB: yes.

[17:19:10] DevB: ok, so just to point out ... one aspect of all this enactment code is error handling.

[17:19:20] DevA: ok.

[17:19:31] DevB: thats obv a big part of the user experience.

[17:19:37] DevA: yeah.

Notes

¹ scuffl is a language used for defining workflows.

The excerpt opens with the observation by DevB that the opportunity exists for the project to extend myExperiment's capabilities and so reach out to a wider user community—'i think with the runner/runnable model we might even be able to support another workflow engine soon'. The discussion between the two developers then turns to the design of an editor that enables users to specify workflow 'inputs'. In particular, following feedback from user YY—'he wasn't happy with the modifications...'—the developers are wrestling with the implications of making its features generic. As DevB notes, a generic approach provides users with full control over what inputs they might specify—'because a basic principle is that there is no limit on the amount of lists of lists yuo can have for one input port'—and would satisfy the requirements of a wider group of users. While this would suit the expert user, DevA observes that it would be more challenging for the casual user, who would probably not be as familiar with the structure for input specification—'but being too generic leads us into the problem where casual users have to create the structure, not just the actual inputs'.

Having run through various candidate solutions, revisited previous occasions when (to their knowledge) the issue has been raised—'that discussion has been had a few times over the project'—and checked their understanding of the current position—'i was under the impression that we were going to give people canned example input sets'—the two developers agree that there is no point in them trying to second guess what users would prefer: the best way to forward is to get user feedback on the current implementation or, as DevB puts it, to '...have the users play with what we have and then see what they ask for'. The excerpt closes with the developers noting an important issue—error handling—that lies in store for them if the enactment capability is to work effectively—'ok, so just to point out ... one aspect of all this enactment code is error handling' and—'thats obv a big part of the user experience.' This illustrates how exploring the full ramifications of decisions may sometimes be sacrificed for the sake of agility.

5.4. Tackling an early service issue

The myExperiment project team is engaged continuously in outreach to new users and domains, keeping a service in use that is also under continuous development (i.e.

'perpetual beta'). In this undertaking it is inevitable (and to some extent intended) that technical issues will emerge that have to be dealt with. With the numbers of users growing quickly in early 2008, the myExperiment server experienced problems in dealing with the load imposed upon it. This event coincided with a new use pattern of concurrent access, as in a Taverna workshop where a group of users did the same thing simultaneously, something for which the codebase at that time was not optimised. Because of its impact on the service, the matter was prioritised and the existing server subsequently replaced by more powerful hardware. In addition, the codebase was optimised using standard web scalability techniques, in a progressive manner (in keeping with the agile nature of the project) in order to meet the environment requirements. This was done with the view that it would be an ongoing piece of work throughout the lifetime of the project. In the following Skype chat extract we show how the myExperiment team responded to and devised a plan to address the service issue.

The problem occurred on two successive days, with the service slowing down. On the first day, these occurrences were reported as they happened via email on the internal mailing list, with an immediate reaction of a team member being to restart a crashed 'service' and the nature and impact of the problem was discussed on the internal email list. On the following day, the service was slow during a Taverna/myExperiment training event (due to the concurrent access), which also was reported promptly by email on the internal mailing list and through an additional personal report to the team by the person in charge of the event. There had been no Skype chats the week before because of other events and staff on leave. Therefore, the Skype chat taking place on the second day of the incident was the first occasion to discuss this problem. In a departure from usual practice for the daily Skype chat, the PM prepared an agenda and circulated this by email an hour before the chat session commenced:

1. Post mortem for both failures
2. Human process for responding to server failure
3. Automated process for responding to server failure
4. Graceful failure modes
5. Service monitoring
6. Testing plans'
7. Actions

Following the project team's established practice, the Skype chat (March 7, 2008) served as a mechanism for combining information exchange and discussion together in the lead up to collective analysis and decision-making. Because of the importance of the problem, the PM ended the chat with a summary of the discussion of each agenda item:

(Abbreviated Skype chat transcription: PM = project manager; Dev = developer)

[18:05:00] PM: just going to summarise against agenda:

[18:05:23] PM: 1. post mortem. we looked at the bodies in the crime scenes, then we presented some theories to inform the investigation

[18:05:46] PM: 2. human process. yes, we'll stay human in the loop for now, but will look at a cronjob for the 3 am solution
[18:06:05] PM: 3. we'll implement health check-in scripts and monitoring by human
[18:06:18] PM: 4. not discussed. probably can improve error reporting
[18:06:31] PM: 5. DevC will work on jmeter
[18:06:41] PM: 6. not discussed, one for next week.
[18:06:45] DevA: I'll process the production log a bit more
[18:07:03] PM: who is going to do health check scripts?
[18:07:10] DevA: i'l do those
[18:07:14] PM: thanks DevA
[18:07:20] PM: ok, we're sorted for now
[18:07:37] DevB: we need to also schedule in the continual performance work—adding caching to other parts of the site (eg groups page) and the eager loading optimisations on models
[18:07:40] PM: Be ready for a roomful of people hitting it just after 9 pm
[18:07:47] DevA: ok
[18:08:04] PM: DevB—yes, and we need to schedule a daily review
[18:08:11] DevB: ok

At the conclusion of the chat, the PM circulated via the internal mailing list a detailed description of outcomes, actions and individuals responsible for them, and a management level summary.

The Skype chat extract shows a mixture of formally structured elements (the agenda circulated beforehand and the summary of actions circulated after) and a lively discussion of the issues (evident here in the, at times, colloquial language—‘we looked at the bodies in the crime scenes’) that, together, lead the team to reach a consensus on what is to be done in a relatively short time.

In this example, we see how the team uses the Skype chat as a forum for planning and decision-making in a way that allows for ideas to be aired and discussed. Plans and decisions are not simply announced by the project manager; instead, they are worked-up collaboratively by the team in and through the chat.

5.5. Managing user expectations

Maintaining the capacity to keep up with user expectations as a project unfolds can be challenging. In this next Skype chat extract (March 11, 2008), the project manager and developers discuss to what extent this might be achieved. They are aware that familiarity with sites such as Facebook is apt to raise users' expectations to a level that the project would find it difficult to emulate, given the limitations of its own resources, such as budget. The extract illustrates how, even a project that is committed to following an agile,

user-centred approach, must not only demonstrate responsiveness to user requirements, but also seek to actively shape them at times.

(Abbreviated Skype chat transcription: PM = project manager; Dev = developer; XX = project team member; ZZ = user)

[17:25:34] PM: then the next question is how long does it take to get a response when you click on Explore or Find

...

[17:26:53] PM: what are the current response times? do they feel acceptable?

[17:27:13] DevA: the “Look at a workflow” response time is good

[17:27:21] PM: XX has asked that we put up a notice saying that due to increasing interest we’re having performance problems and we are looking at improvements

[17:27:34] DevB: note: “look at workflow” has been changed to “find workflows” based on user feedback

[17:27:37] PM: so we’ve done an announcement for existing users

[17:27:45] PM: but we haven’t yet put this on the home page

[17:27:52] PM: i would like people’s views on this please

[17:28:01] DevA: yeah—I was just doing that on the live site

[17:28:05] DevB: new users can still see the announcements when they go to/ home ... but yes, its not on front page

[17:28:30] DevE: i personally dont think it should be

[17:28:37] PM: DevB and i discussed it on the phone earlier and decided to mull it over during the day—question is how best to word it positively

[17:29:04] PM: that was my original reaction too

[17:29:08] DevF: how long would it be a problem for?

[17:29:14] PM: forever

...

[17:29:25] PM: stepping back....

[17:29:42] PM: this is a proof-of-concept site, not a 20 million pound facebook site

[17:30:09] DevF: then perhaps that’s the thing to put there. that the local deployment stuff is important too

[17:30:16] DevB: to put in perspective—FB are spending \$200mill on infrastructure in next few years

[17:30:24] DevA: also, the fact that the front page has completely changed is enough for existing users to know that a big change has happened. they’ll probably reevaluate the site’s speed because of that

[17:30:27] DevA: (if they see the front page at all)

[17:30:42] DevB: true

[17:30:44] DevB: perceived speed

[17:31:02] DevB: ZZ has already mentioned that he is noticing better speeds when clicking on the tabs (this is without me even asking him)

[17:31:13] DevA (nods)¹

[17:31:20] PM: i think we ahve to contineu to work on performacne

[17:31:25] PM: but we also need some expection management

[17:31:33] DevA: are there any current known performance problems?

[17:31:45] PM: thing is, impressing people with it is part of our sustainability plan!

Notes

¹ () denotes use of a skype emoticon.

The discussion has been triggered by continuing concerns that users may be experiencing poor performance—‘XX has asked that we put up a notice saying that [...] we’re having performance problems and we are looking at improvements’. A few days previously (see section 5.4), dramatically deteriorating response times had demanded rapid and decisive intervention. Today, the extent of the problem is much less clear—‘what are the current rsppnse times? do they feel acceptable?’; ‘the “Look at a workflow” response time is good’—and it seems that some measures already taken may have met with some success—‘ZZ has already mentioned that he is noticing better speeds when clicking on the tabs’. Despite the conflicting evidence, the PM declares that the team must not let up in its efforts to improve performance—‘i think we ahve to contineu to work on performacne’.

At the same time, the PM acknowledges that, because of limited resources and specific project goals—‘this is a proof-of-concept site, not a 20 million pound facebook site’—the team will have (at least as a stopgap measure) to resort to try to influence user expectations—‘but we also need some expection management’. This, of course, is not without risk—‘thing is, impressing people with it is part of our sustainability plan!’—and so has to be done with great care—‘question si hlow best to word it [the announcement] positively’. Their approach is to attribute the problem to the site’s success—‘due to increasing interest’. However, it may be that users will re-calibrate their own expectations—‘the fact that the front page has completely changed is enough for existing users to know that a big change has happened. they’ll probably reevaluate the site’s speed because of that’.

5.6. Keeping the project on track

Throughout this paper, we have attempted to document and detail how agile approaches to project management (in developing a VRE), deliver flexibility in the software development process. This flexibility allows the project to accommodate important aspects of user needs. We suggest this approach particularly makes sense in a context where user requirements are complex—not given, but co-emergent with the capabilities of the software and services being created. We also contrast methodologically formulaic approaches to software development against the more

flexible, responsive and improvisational character of agile approaches and illustrate how we can see such flexibility exhibited in the fieldwork material. Nevertheless, the relationship of this flexibility to the more planful aspects of project management remain, perhaps, somewhat opaque. While the formal and informal structuring of day to day project work is readily apparent in the fieldwork data presented so far, we, perhaps, get less of an insight or clear view of its relation to the ‘higher level’ management of the project. The next example aims to bring some visibility to this set of issues by focusing on a discussion about the core meaning of myExperiment to the team.

In the following Skype chat extract (February 18, 2008), we see how the issue examined in section 5.3 concerning user requirements for the specification of workflow inputs has triggered discussion about a more fundamental question, i.e. whether myExperiment takes a ‘service centric’ or ‘data centric’ view of workflows and the implications for the project of switching from the former to the latter at this stage.

(Abbreviated Skype chat transcription: PM = project manager; Dev = developer)

[17:40:21] DevA:—we have a generic taverna input thingy in a web page

[17:40:44] ... but it’s not obvious to casual users what the structure of a workflow’s input should be

...

[17:41:17] DevA: we’d really need to be able to let people who submit workflows describe the list/string structure of the inputs so that users don’t have to create the structure themselves

[17:41:30] DevD: i see

[17:41:39] DevB: yeap

[17:41:54] ... thats a big client side piece—defining the application front end to a workflow

[17:42:13] DevA: and also taverna specific in the lists/strings behaviour

[17:42:18] DevB: anyway, by the sounds of it... i think this is going to move away from myexperiment for the time being... since myExp isnt funded for this

[17:42:25] ... yeap

...

[17:43:04] DevA: i thought myexperiment would deal with the user facing side of it though

[17:43:08] DevB: doyou want to discuss the ideas?

[17:43:21] ... DevA—the problem is that its a big piece

[17:43:35] ... too big for our shoes just yet

[17:43:45] DevD: yes it’s very related to whether we are data or service centric on myexp

[17:43:54] DevB: hmmm

[17:43:54] DevD: atm we are service centric

[17:44:13] ... but all the time we've been trying to become data centric in taverna, in the past years

The myExperiment project plan has a formal rendering as workpackages, milestones and so on. Such formal project plans are a recognisable prescription for accountably meeting the project's contractual obligations—that something (a VRE, a website, etc.) must be built and to some schedule. So, the issue that we examine in this extract concerns how accountably “keeping the project on track” is achieved when there has to be flexibility in terms of what the project might actually deliver. In particular, we see how the team orientates to the challenge of deciding if a re-focusing of the project should be attempted—‘whether we are data or service centric on myexp’. In the event, the decision is that this is simply not feasible—‘too big for our shoes just yet’—with the resources currently available and is out of scope—‘myExp isnt funded for this’—and so is best postponed until a later time (and possibly a new round of funding).

6. Discussion

Our initial interest was primarily in understanding the character of myExperiment as a ‘project’—how it actually ‘gets done’ and how the obvious, visible orderliness of the project is practically achieved while the team attends to the challenges of achieving embedding within its user community. We saw in the excerpts of Skype chats examples of how the myExperiment team responded to a range of issues that typically arise in the course of software project work, i.e.: prioritising development tasks; deliberating and deciding on technical strategy; making use of user feedback to resolve an interface design question; diagnosing and fixing a potentially damaging failure; managing user expectations; and reviewing the direction and scope of the project.

What we reveal in these excerpts is not that the myExperiment project team is successful in resolving each and every issue ‘once and for all’ (indeed, we see in these excerpts how issues such as dealing with user expectations and performance problems are resolved only to reappear later in different guises). Instead, we see how the team strives (subject to all the usual contingencies) to resolve issues as soon as possible after they arise. The team aims to maintain the myExperiment site in working order, without sacrificing its capacity to keep it moving forward and broadly in line with (sometimes rapidly) evolving user needs and expectations. That is, facing real world, real time, challenges the team seeks ‘satisficing’ solutions to their problems (Simon 1956 1969). Drawing on our previous studies of software projects, we would place the myExperiment project somewhere in a “middle-ground between the conventional, highly formalised software engineering and ‘hacking’ that inscribes collective improvisation” (Mackenzie and Rouncefield 2002).

Viewing the myExperiment project as a practical, ongoing achievement, and concentrating on the everyday, mundane aspects of keeping a project going, we have placed a particular emphasis on various kinds of ‘ordering work’ that occurs at a number of levels and draw attention to the various forms of communication—through email, Skype chat, etc. Given the distributed nature of the myExperiment team, we have seen how the project management attaches considerable importance to coordination, communication, accountability and awareness. We have also seen how the team seeks to perform a careful balancing act between planning and agility, between control and flexibility, in order to keep the direction of the project open and to capitalise on a continuous stream of users’ requests for enhancements, while, at the same time, preserving a sufficient degree of oversight and control on the part of the project management.

Our study has explicated some of the everyday, mundane work of agile, distributed software project management, pointing, in particular, to some details as to how the project is effectively micro-managed. The aim of such micro-management is to try and ensure that the project remains on track with development and testing, while simultaneously providing a pilot service, and trying to ensure user engagement such that the project is able to evolve in line with changing user requirements. We have also provided some initial examples of ways in which various messaging tools and aspects of ‘virtual collaboration’ create, nurture or sustain particular arrangements for the everyday management of projects.

Our study also shows the continuing relevance of Button and Sharrock’s (1996) framework for understanding how project work gets done; the importance of ‘phasing’ and the orderly handling of tasks; the everyday work involved in ensuring that project members ‘orient to the project as a totality’, that they keep in mind the eventual goals and requirements of the users and measure progress against this, and so on. The work to stay organized is a constituent part of any project:

“getting it done involves the determination of such matters as how much work there is to be done, how long it will take, how many must be involved, how much time is available, how those involved are to combine their activities to carry the work through, and how they are to ensure that their activities will remain coordinated and synchronised over its course, what is to be done in various eventualities, who will make the judgement as to whether the work has been done satisfactorily and what it will take to satisfy them.” (Sharrock and Anderson 1993:161).

Some aspects of Button and Sharrock’s framework appear less prominently, however. For example, and in keeping with the principles of agile methods, ‘documentation’ and, specifically, ensuring that documentation gets done, feature less heavily. In addition, the overall emphasis on users (what they want and their likely reactions) feature far more strongly in the project discussions, and everyday

email and Skype chat. Again, this is in keeping with agile principles, while also confirming Woolgar's (1991) general conclusion, that a substantive part of any software project involves reasoning, speculating, about what users might do with the system.

Other features of project work appear far more prominently in our study than they do in Button and Sharrock's (*ibid*) original formulation. What is especially noticeable is how the more affective aspects of project work, various forms of 'emotional labour' (Hochschild 1983) concerning orientations to both colleagues and, importantly, users, are far more explicit than in Button and Sharrock's work. This is where the findings of Ramesh et al. (2006) seem to us to be especially pertinent, in particular, their stress on the importance of communication, control and, above all, the building of trust, team morale and cohesion in distributed, agile software development.

The first of these features concerns the various forms of managerial and leadership work required to keep the project on track, particularly a project that relies so heavily on technology (Skype chat, email etc.) to motivate and manage a virtual team. This provides both an obstacle to be overcome (the difficulties of managing at a distance) and an opportunity to be exploited (the provision of an obvious 'audit trail' as a by-product of using Skype chat and email to communicate). In common with Ramesh et al. (*ibid*), we observe in the myExperiment project the necessity of 'constant communication' via email, Skype chat, etc. in the team's attempt to guarantee the orderly coordination and management of the project.

A second feature concerns a set of activities to do with the communal, the affective, aspect of project work—motivating the work while keeping both the customer and the team satisfied. In Ramesh et al.'s (*ibid*) case studies, trust and team morale building was supported through various formal mechanisms such as partner site visits. While this also featured in the myExperiment project, far more notable, in our opinion, was recognising how trust and morale was built and displayed through the daily interaction of the Skype chats. What we document is, in effect, an important aspect of the daily life of the project. This is displayed in the frequent supportive comments of the project manager, the 'gr8's', the 'you're doing the right thing' and so on, as well as the relaxed sharing of personal, social information—e.g. 'I've handed in my thesis the beers are on me'.

Finally, we note the relevance of De Roure and Goble's (2008) 'Six Principles of Design', with their particular emphasis on fostering user engagement, for understanding this aspect of Web 2.0 project work. The importance of fitting in with current user practices, of not second-guessing user requirements, of incentivising and empowering users, of delivering 'jam today and more jam tomorrow' are evident not just as interesting slogans but as everyday features of project work and discussions. This emphasis on users also has important, necessary temporal aspects, since time is of constant relevance (*when* things can be done, *how long* it takes to respond to queries, *how long* it takes to repair bugs,

etc.). Consequently this interaction between time constraints and user demands means that project work appears to be essentially a ‘satisficing’ matter: “*procedures that find good enough answers to questions whose best answers are unknowable*” (Simon 1969: 28). It is in this sense that the myExperiment project team can be seen to be following an agile methodology; favouring rapid responses to change over the careful following of a ‘plan’ and working software over comprehensive documentation—features that resonate throughout the project.

Although there is not enough room to develop this analysis here, another, and related, area of interest, given the emphasis on social networking in the myExperiment VRE, concerns the evolving nature of the ‘community’ involved in myExperiment, with some interesting clues as to exactly how a user community is built and sustained. In particular, what we see at work in this early analysis of the emails and Skype logs, etc. is how the defining features of community, the key features of boundaries, relationships, obligations and change, are rapidly identified, negotiated and renegotiated in public throughout the course of the project. We have already pointed to the importance of temporal features of project work, but the rapidity with which Web 2.0 tools can be developed, deployed, tested and redesigned appears to have the potential to impact significantly on a community, perhaps especially the hybrid community of software engineers, computer scientists, natural scientists and ‘professional end-user developers’ (Segal 2005) observed on this project.

7. Conclusions

In this paper, we have examined the different ways in which the myExperiment project team orientate themselves to project work. We document how the use of a range of messaging tools allows team members located at different places to have frequent interaction and how the team’s preference for using Skype chat over talk furnishes them with a simple collaboration space and a record of each meeting. In these ways, the team members are able to: coordinate complex and interdependent development work; to both track progress and to be seen to be tracking it; to tackle problems; to manage user engagement so as to ensure that the team is furnished with a continuing supply of requirements; and to build and maintain trust, team morale and cohesion.

What we note is how these different kinds of interaction come into play to deliver the characteristic of a ‘perpetual beta’, that is, to deliver value quickly to users, while maintaining the project in a state of being a ‘doable’ enterprise. We have also seen that this necessarily calls for the team to adopt a ‘satisficing’ strategy that takes into account the need to act while under the constraints of time, working with imperfect knowledge and volatile requirements, while maintaining user commitment at an adequate level.

Our study of the myExperiment project furnishes further evidence that distributed software development can be done in an agile way, though not, of course, that it will be work for each and every project. For example, we have not had the opportunity to see how the myExperiment project team would cope with a much larger user base and the prospect of dealing with escalating demands for new features; nor does our case study provide proof that such an agile management approach can be successfully scaled up for use in a much bigger project.

Nevertheless, we would argue that the use of various communication and structuring devices by the myExperiment project team holds some useful lessons for the eResearch community about how to respond to the challenges and opportunities in the daily, lived experience of agile project management practice. Our findings suggest that the essence of agile project management is to be found in the ongoing struggle for a balance between the seemingly contradicting and conflicting agendas of flexibility and planfulness, and improvisation and orderliness. In turn, we argue that it is the maintenance of this balance that provides the foundations for the meaningful and continuous user engagement essential to tackling the challenges of embedding eResearch applications successfully.

Notes

- ¹ The UK Office of Science and Technology e-Infrastructure Working Group defines a VRE as “a set of online tools, systems and processes interoperating to facilitate or enhance the research process within and without institutional boundaries.” See Borda et al. (2006).
- ² http://www.jisc.ac.uk/programme_vre.html
- ³ A scientific workflow is a digital artefact that enables the composition of multiple, individual steps in a research process into a single computational entity. In this form, research processes can be automated and, just as importantly, easily shared with and re-used by others.
- ⁴ <http://oreilly.com/pub/a/web2/archive/what-is-web-20.html?page=5> (Retrieved on December 4, 2010)
- ⁵ http://wiki.myexperiment.org/index.php/Main_Page
- ⁶ See <http://www.youtube.com/watch?v=x83pzMMw7lk> (Retrieved on December 4, 2010)
- ⁷ The myGrid project is home to a range of individual projects, including myExperiment and Taverna, that share a focus of developing tools to “help e-Scientists get on with science and get on with scientists.” See http://wiki.myexperiment.org/index.php/Main_Page www.mygrid.org.uk/
- ⁸ CVS (Concurrent Versioning System) is a tool that allows software project teams to keep track of changes in program files.
- ⁹ The term ‘hackfest’ refers to an intensive, collaborative coding session, designed to accelerate development. The myExperiment hackfests take place every month, last between 2 and 3 days, and focus on getting key requirements translated into code.
- ¹⁰ A perception voiced by many developers of e-Research applications is that grid technical standards are too complex and difficult to use (see, e.g. Chin and Coveney 2004).

References

- Abrahamsson, P., Wärsta, J., Siponen, M. and Ronkainen, J. (2003). New directions on agile methods: A comparative analysis. In: *Proceedings of the 25th International Conference on Software Engineering*, Portland, May. IEEE.
- Beck, K. (2000). *Extreme programming explained: Embracing change*. Addison Wesley.
- Boehm, B. W. (1988). A spiral model of software development and enhancement. *IEEE Computer*, 21, 61–72.
- Borda, A., Careless, J., Dimitrova, M., Fraser, M., Frey, J., Hubbard, P., et al. (2006). *Report of the Working Group on Virtual Research Communities for the OSTe-Infrastructure Steering Group*. London, UK: Office of Science and Technology.
- Button, G., & Sharrock, W. (1994). Occasioned practices in the work of software engineers. In M. Jirotko & J. Goguen (Eds.), *Requirements engineering: Social and technical issues*. London: Academic.
- Button, G., & Sharrock, W. (1996). Project work: The organisation of collaborative design and development in software engineering. *Computer Supported Cooperative Work: The Journal of Collaborative Computing*, 5, 369–386.
- Chin, J. and Coveney, P. (2004). Towards tractable toolkits for the Grid: a plea for lightweight, usable middleware. UK e-Science Technical Report Series, National e-Science Centre. Available at http://www.nesc.ac.uk/technical_papers/UKeS-2004-01.pdf (Retrieved on December 1, 2010).
- Crabtree, A., Nichols, D., O'Brien, J., Rouncefield, M., & Twidale, M. (2000). Ethnomethodologically informed ethnography and information system design. *Journal of the American Society for Information Science*, 51(7), 666–682.
- De Roure, D., & Goble, C. (2008). Six principles of software design to empower scientists. *IEEE Software*, 26(1), 88–95.
- De Roure, D., Goble, C. and Stevens, R. (2007). Designing the myExperiment virtual research environment for the social sharing of workflows. In: *Proceedings of the Third IEEE International Conference on e-Science and Grid Computing* (pp. 603–610), Bangalore, India, 10–13 December.
- Dyba, T., & Dingsoyr, T. (2008). Empirical studies of agile software development: A systematic review. *Information and Software Technology*, 50(9–10), 833–859.
- Fowler, M. and Highsmith, J. (2001). The agile manifesto. *Software Development Magazine*. August. Available at <http://www.sdmagazine.com/documents/s=844/sdm0108a/0108a.htm> (Retrieved May 8, 2009).
- Gil, Y., Deelman, E., Ellisman, M., Fahringer, T., Fox, G., Gannon, D., et al. (2007). Examining the challenges of scientific workflows. *IEEE Computer*, 40(12), 24–32.
- Glaser B. and Strauss, A. (1967). *Discovery of grounded theory: strategies for qualitative research*. Sociology Press.
- Hey, T and Trefethen, A. (2003). The data deluge: An e-Science perspective. In: Berman, F., Fox, G. and Hey, A. (Eds.), *Grid computing—making the global infrastructure a reality*. Wiley.
- Hine, C. (2000). *Virtual ethnography*. Sage.
- Hochschild, A. (1983). *The managed heart: Commercialisation of human feeling*. Berkeley: University of California Press.
- Hole, S., & Moe, N. B. (2008). A case study of coordination in distributed agile software development. *Communications in Computer and Information Science*, 16(5), 189–200.
- Hughes, J. A., King, V., Rodden, T. and Andersen, H. (1994). Moving out from the control room: Ethnography in system design. In: *Proceedings of CSCW'94*. Chapel Hill: ACM.
- Mackenzie, A. and Rouncefield, M. (2002). How ‘hacking’ hides a project: from software engineering to open source and back again. Appendix C. Dependability Issues in Open Source Software DIRC Project Activity 5 Final Report. University of Newcastle on Tyne.
- Morris, J. (2006). Software product management and the endless beta. Available at http://jimorris.blogspot.com/2006_08_01_jimorris_archive.html (Retrieved April 9, 2008).

- Procter, R., Williams, R., Stewart, J., Poschen, M., Snee, H., Voss, A., et al. (2010). Adoption and use of Web 2.0 in scholarly communications. *Philosophical Transactions of the Royal Society A, special issue on e-Science*, 368(1926), 4039–4056.
- Ramesh, B., Cao, L., Mohan, K., & Xu, P. (2006). Can distributed software development be agile? *Communications of the ACM*, 49(10), 41–46.
- Randall, D., Harper, R. and Rouncefield, M. (2007). *Fieldwork for Design: Theory and practice*. Kluwer.
- Royce, W.W. (1987 [1970]). Managing the development of large software systems: Concepts and techniques. In: *Proceedings of the 9th International Conference on Software Engineering* (pp. 328–338). Monterey, CA.
- Russo, N. L., & Stolterman, E. (2000). Exploring the assumptions underlying information systems methodologies: Their impact on past, present and future ISM research. *Information Technology & People*, 13(4), 313–327.
- Schwaber, K., & Beedle, M. (2002). *Agile software development with scrum*. NJ: Prentice-Hall.
- Segal, J. (2005). Two principles of end-user software engineering research. In: *Proceedings of the first workshop on end-user software engineering*. St Louis: ACM.
- Sharp, H., Robinson, H., & Petre, M. (2009). The role of physical artefacts in agile software development: Two complementary perspectives. *Interacting with Computers*, 21(1–2), 108–116.
- Sharrock, W., & Anderson, B. (1993). Working towards agreement. In G. Button (Ed.), *Technology in working order* (pp. 149–161). London: Routledge.
- Simon, H. A. (1956). Rational choice and the structure of the environment. *Psychological Review*, 63(2), 129–138.
- Simon, H. (1969). *The sciences of the artificial*. Cambridge: MIT.
- Sommerville, I. (2001). *Software engineering* (6th ed.). Harlow: Pearson Education & Addison Wesley.
- Suchman, L. (1994). Working relations of technology production and use. *Computer Supported Cooperative Work Journal*, 2(1–2), 21–39.
- Suchman, L. (1995). Making work visible. *Communications of the ACM*, 38(9), 33–35.
- Voss, A., & Procter, R. (2009). Virtual research environments in scholarly work and communications. Special issue on Virtual Research Environments. *Library Hi Tech Journal*, 27(2), 174–190.
- Wastell, D. G. (1996). The fetish of technique: Methodology as a social defence. *Information Systems Journal*, 6, 25–40.
- Woolgar, S. (1991). Configuring the user: The case of usability trials. In J. Law (Ed.), *A sociology of monsters. Essays on power technology and domination* (pp. 58–100). London: Routledge.